

```
benchmark.cpp

Starting Benchmarks using omp_get_wtime()...
[DONE] Merge Sort
[DONE] Matrix Multiplication
[DONE] BFS

All benchmarks executed successfully using omp_get_wtime()! Output exported to:
F:\BRACU\CSE706 - Parallel Algorithm\Parallel_Computing_Project\benchmark_results.csv

collapse_benchmark.cpp

Running Matrix Multiplication Collapse Benchmark...

Matrix  Threads  Normal parallel for  collapse(2)  Faster
-----
256    1    679.999828 ms    848.999977 ms  Normal
256    2    411.000013 ms    392.999887 ms  Collapse(2)
256    4    233.999968 ms    226.999998 ms  Collapse(2)
256    8    130.000114 ms    128.999949 ms  Collapse(2)
512    1   4372.999907 ms   6392.999887 ms  Normal
512    2   3378.999949 ms   3419.000149 ms  Normal
512    4   2331.999779 ms   2236.999989 ms  Collapse(2)
512    8   1194.000006 ms   1196.000099 ms  Normal
1024   1   43437.000036 ms  46072.999954 ms  Normal
1024   2   27975.999832 ms  28115.000010 ms  Normal
1024   4   18015.999794 ms  17832.999945 ms  Collapse(2)
1024   8    9400.000095 ms   9437.999964 ms  Normal

[SUCCESS] Benchmark results saved to CSV: data\benchmark_results\collapse_results.csv

controlled_matrix_benchmark.cpp

=====
CONTROLLED MATRIX MULTIPLICATION BENCHMARK (Identical Input Data)
=====

>>> Generating Controlled Master Input Matrices A and B (256x256)... Done.

Matrix  Threads  Sequential  Static    Dynamic    Collapse(2)
-----
256    1    809.000015 ms  661.999941 ms  784.000158 ms  597.999811 ms
256    2    809.000015 ms  465.999842 ms  460.000038 ms  465.000153 ms
256    3    809.000015 ms  319.000006 ms  307.999849 ms  319.000006 ms
256    4    809.000015 ms  251.000166 ms  282.999992 ms  221.999884 ms

>>> Generating Controlled Master Input Matrices A and B (512x512)... Done.

Matrix  Threads  Sequential  Static    Dynamic    Collapse(2)
-----
```

```
512 1 6111.999989 ms 9383.999825 ms 7073.000193 ms 6431.999922 ms
512 2 6111.999989 ms 3729.000092 ms 3820.999861 ms 3741.999865 ms
512 3 6111.999989 ms 2687.000036 ms 2556.999922 ms 2653.000116 ms
512 4 6111.999989 ms 2209.000111 ms 2206.999779 ms 2128.000021 ms
```

[SUCCESS] Controlled benchmark results logged to: data\benchmark_results\controlled_matrix_results.csv

full_benchmark.cpp

```
=====
WEEK 3 MAJOR BENCHMARK EXPERIMENT (3 Sizes x 4 Threads = 12 Runs per Variant)
=====
```

```
>>> Generating Matrix Data N=256 (256x256)... Done.
>>> Executing Ground Truth Sequential Baseline... 613.00 ms
```

Matrix	Threads	Sequential	Static	Dynamic	Collapse(2)	Verified
256	1	612 ms	559 ms	591 ms	495 ms	PASSED
256	2	612 ms	460 ms	364 ms	373 ms	PASSED
256	4	612 ms	289 ms	285 ms	284 ms	PASSED
256	8	612 ms	189 ms	159 ms	158 ms	PASSED

```
>>> Generating Matrix Data N=512 (512x512)... Done.
>>> Executing Ground Truth Sequential Baseline... 5852.00 ms
```

Matrix	Threads	Sequential	Static	Dynamic	Collapse(2)	Verified
512	1	5851 ms	5461 ms	5723 ms	4891 ms	PASSED
512	2	5851 ms	3369 ms	3251 ms	3463 ms	PASSED
512	4	5851 ms	2344 ms	2298 ms	2275 ms	PASSED
512	8	5851 ms	1279 ms	1328 ms	1372 ms	PASSED

```
>>> Generating Matrix Data N=1024 (1024x1024)... Done.
>>> Executing Ground Truth Sequential Baseline... 47559.00 ms
```

Matrix	Threads	Sequential	Static	Dynamic	Collapse(2)	Verified
1024	1	47559 ms	46868 ms	49139 ms	46720 ms	PASSED
1024	2	47559 ms	28425 ms	26924 ms	28741 ms	PASSED
1024	4	47559 ms	18789 ms	17985 ms	18138 ms	PASSED
1024	8	47559 ms	9973 ms	9896 ms	10082 ms	PASSED

```
=====
[SUCCESS] Week 3 full benchmark complete. All 12 test points passed verification.
Dataset written to: data\benchmark_results\week3_full_benchmark.csv
=====
```

matrix_schedule_benchmark.cpp

```
=== Matrix Multiplication Schedule Benchmark (512x512) ===
```

--- Thread Count: 1 ---
Threads: 1 | Schedule: static | Time: 5444 ms
Threads: 1 | Schedule: dynamic | Time: 5319 ms
Threads: 1 | Schedule: guided | Time: 5222 ms

--- Thread Count: 2 ---
Threads: 2 | Schedule: static | Time: 3684 ms
Threads: 2 | Schedule: dynamic | Time: 3378 ms
Threads: 2 | Schedule: guided | Time: 3968 ms

--- Thread Count: 3 ---
Threads: 3 | Schedule: static | Time: 2972 ms
Threads: 3 | Schedule: dynamic | Time: 2858 ms
Threads: 3 | Schedule: guided | Time: 2852 ms

--- Thread Count: 4 ---
Threads: 4 | Schedule: static | Time: 2316 ms
Threads: 4 | Schedule: dynamic | Time: 2345 ms
Threads: 4 | Schedule: guided | Time: 2331 ms

test_race_conditions.cpp

Testing Parallel Merge Sort for Race Conditions (10 runs, 4 threads)...

Run 1 -> Correct
Run 2 -> Correct
Run 3 -> Correct
Run 4 -> Correct
Run 5 -> Correct
Run 6 -> Correct
Run 7 -> Correct
Run 8 -> Correct
Run 9 -> Correct
Run 10 -> Correct

SUCCESS: All 10 runs sorted correctly. No race conditions detected!

verified_matrix_benchmark.cpp

=====

STRICT VERIFICATION & BENCHMARKING (Sequential == Parallel Check)

=====

>>> Initializing Master Input Matrices A and B (256x256)... Done.
>>> Running Sequential Baseline... 726 ms

Matrix	Threads	Sequential	Static	Dynamic	Collapse(2)	Verified
256	1	726.000071 ms	638.999939 ms	628.000021 ms	552.000046 ms	PASSED
256	2	726.000071 ms	471.999884 ms	417.999983 ms	452.999830 ms	PASSED

[CORRUPTION DETECTED] Collapse(2) output mismatch at N=256, threads=3!

bfs_parallel.cpp

BFS (Parallel Level-Synchronous) Time: 1158.64 ms

matrix_multiplication_parallel.cpp

Matrix Multiplication (Parallel For) Time: 1635.32 ms

bfs_sequential.cpp

Enter the number of vertices: 6

Enter the number of edges: 5

Enter 5 edges (u v pairs, 0-indexed):

0 1

0 2

1 3

1 4

2 5

Enter the starting source node for BFS: 0

BFS Traversal starting from node 0: 0 1 2 3 4 5

merge_sort_sequential.cpp

Enter the number of elements: 5

Enter 5 space-separated elements: 25 4 85 22 6

Original array: 25 4 85 22 6

Sorted array: 4 6 22 25 85

matrix_multiplication_sequential.cpp

Enter rows and columns for Matrix A (R1 C1): 3 3

Enter rows and columns for Matrix B (R2 C2): 3 3

Enter elements of Matrix A (3x3):

1 2 3

4 5 6

7 8 9

Enter elements of Matrix B (3x3):

11 22 33

44 55 66

77 88 99

Resultant Matrix C (3x3):

330 396 462

726 891 1056

1122 1386 1650